

Pointer

A pointer is a variable that stores the memory address of another variable. This means that instead of holding a data value (like an integer or character), a pointer holds the location in memory where that value is stored. Pointers can point to various data types, including integers, characters, and even functions.

Declaration of Pointers

To declare a pointer in C, you use the asterisk (*) symbol before the pointer variable name.

The general syntax is:

```
data_type *pointer_name;
```

Examples of Pointer Declarations

- ***Single Pointer Declaration:*** `int *ptr; // Declares a pointer variable named ptr that can hold the address of an integer`
- ***Multiple Pointer Declarations:*** `int *p1, *p2; // Declares two pointer variables p1 and p2, both pointing to integers`
- ***Pointer to a Constant:*** `const int *ptrConst; // Declares a pointer to a constant integer`

Initialization of Pointers

After declaring a pointer, it must be initialized with the address of a variable using the address-of operator (&).

The syntax for initialization is:

```
pointer_name = &variable_name;
```

Example of Pointer Initialization

```
int myAge = 43; // An integer variable
```

```
int *ptr = &myAge; // ptr is initialized to hold the address of myAge
```

Common mistakes when working with pointers

Suppose, you want pointer pc to point to the address of c. Then,

```
int c, *pc;
```

```
// pc is address but c is not
```

```
pc = c; // Error
```

```
// &c is address but *pc is not  
*pc = &c; // Error
```

```
// both &c and pc are addresses  
pc = &c; // Not an error
```

```
// both c and *pc are values  
*pc = c; // Not an error
```

Here's a complete program demonstrating declaration and initialization of pointers:

```
#include <stdio.h>  
int main() {  
    int myAge = 43; // Declare an integer variable  
    int *ptr ;  
    ptr= &myAge; // Initialize pointer with the address of myAge  
  
    printf("Value of myAge: %d\n", myAge); // Output: 43  
    printf("Address of myAge: %p\n", &myAge); // Output: Address of myAge  
    printf("Address of myAge: %p\n", ptr); // Output: Address of myAge  
    printf(" ptr points to value: %d\n", *ptr); // Dereference ptr to get value: 43  
    return 0;}
```

Pointer Operators

C provides several operators for working with pointers, primarily the address-of operator (&) and the dereference operator (*).

1. Address-of Operator (&)

This operator is used to obtain the memory address of a variable.

Example:

```
int x = 10,*ptr;  
ptr = &x; // ptr now holds the address of x
```

2. Dereference Operator (*)

This operator is used to access or modify the value stored at the address pointed to by the pointer.

Example:

```
int y = *ptr; // y now holds the value at the address stored in ptr (which is 10)  
*ptr = 20; // Changes the value of x to 20 through ptr
```

Pointer arithmetic

Pointer arithmetic in C is a way to manipulate pointers by performing arithmetic operations like addition, subtraction, and comparison. Since pointers store memory addresses, pointer arithmetic allows you to move through memory locations and access data structures like arrays more efficiently.

1. Pointer Addition

When you increment a pointer, it points to the next element of its type, not just the next byte in memory. The size of the increment depends on the type of the pointer.

Example:

```
int arr[] = {10, 20, 30, 40};  
int *ptr = arr; // ptr points to the first element of the array (arr[0])
```

```
ptr++; // Moves to the next integer, arr[1]  
printf("%d\n", *ptr); // Outputs 20
```

In this example:

ptr++ increments the pointer by sizeof(int), which is typically 2 bytes, so it now points to arr[1].

2. Pointer Subtraction

Just as you can increment a pointer, you can also decrement it to move to a previous element. The pointer will decrease by the size of the type it points to.

Example:

```
int arr[] = {10, 20, 30, 40};  
int *ptr = &arr[3]; // ptr points to arr[3], which is 40
```

```
ptr--; // Moves to arr[2]  
printf("%d\n", *ptr); // Outputs 30
```

In this example:

Ptr-- decrements the pointer by sizeof(int), which is typically 2 bytes, so it now points to arr[2], which is 30.

3. Pointer Difference (Subtraction of Pointers)

You can subtract one pointer from another, but both pointers must point to elements within the same array (or memory block). The result is the number of elements between the two pointers, not the difference in memory addresses.

Example:

```
int arr[] = {10, 20, 30, 40};  
int *ptr1 = &arr[1]; // Points to arr[1]  
int *ptr2 = &arr[3]; // Points to arr[3]
```

```
diff = ptr2 - ptr1; // Difference in terms of number of elements  
printf("%td\n", diff); // Outputs 2 (arr[3] - arr[1])
```

In this example:

ptr2 - ptr1 gives the number of elements between ptr1 and ptr2, which is 2 (since arr[3] is two elements after arr[1]).

4. Pointer Comparison

You can compare pointers using relational operators (<, >, ==, etc.) to see their relative positions in memory. Typically, this is done to check if two pointers point to the same element or if one pointer is before or after another in an array.

Example:

```
int arr[] = {10, 20, 30, 40};  
int *ptr1 = &arr[1]; // Points to arr[1]  
int *ptr2 = &arr[2]; // Points to arr[2]
```

```
if (ptr1 < ptr2) {  
    printf("ptr1 is before ptr2\n"); // Outputs this because ptr1 points to arr[1] and ptr2  
    points to arr[2]  
}
```

The key idea behind pointer arithmetic is that when you increment or decrement a pointer, C automatically considers the size of the type it points to.

This means:

ptr++ does not just add 1 to the address, it adds sizeof(*ptr) bytes.
Similarly, ptr-- subtracts sizeof(*ptr) bytes.

For example:

If ptr is a pointer to an int, and an int typically takes 2 bytes of memory, then ptr++ will increment ptr by 2 bytes (not just 1 byte).

If ptr is a pointer to a char, then ptr++ will increment ptr by 1 byte, since a char is 1 byte in size.

Example: Pointer Arithmetic with Arrays

```
#include <stdio.h>

int main() {
    int arr[] = {5, 10, 15, 20};
    int *ptr = arr;

    // Access elements using pointer arithmetic
    printf("First element: %d\n", *ptr);    // 5
    ptr++;
    printf("Second element: %d\n", *ptr);    // 10
    ptr++;
    printf("Third element: %d\n", *ptr);    // 15
    ptr++;
    printf("Fourth element: %d\n", *ptr);    // 20
    return 0;}
```

In this example:

ptr++ moves the pointer to the next element in the array arr[].

We dereference the pointer (*ptr) to access the value stored at that address.

Returning multiple values from a function using pointers:

In C and C++, you can return multiple values from a function using pointers by passing address of variables as arguments and having the function modify the values directly through those pointers. This is a common technique since C/C++ functions can only return a single value. Here's an example of how you can do this:

Example:

```
#include <stdio.h>

void returnMultipleValues(int *a, int *b, int *c) {
    *a = 10;
    *b = 20;
    *c = 30;
}

int main() {
    int x, y, z;
```

```
// Call the function and pass the addresses of x, y, and z
returnMultipleValues(&x, &y, &z);

// Print the values after the function modifies them
printf("x = %d, y = %d, z = %d\n", x, y, z);

return 0;}
```

Explanation:

- The function returnMultipleValues takes three pointer arguments (int *a, int *b, int *c).
- Inside the function, it dereferences these pointers to modify the values stored at the addresses passed from main.
- In main, we declare three variables (x, y, z) and pass their addresses to the function using the address-of operator (&).
- After the function call, the values of x, y, and z are modified as expected, and we print them.

Output:

x = 10, y = 20, z = 30

Example: Swapping Two Numbers Using returning multiple values using pointer

```
#include <stdio.h>
```

```
// Function to swap two integers using pointers
```

```
void swap(int *a, int *b) {
    int temp = *a; // Store the value of a in temp
    *a = *b;       // Assign the value of b to a
    *b = temp;     // Assign the value of temp (old a) to b
}
```

```
int main() {
    int x = 5, y = 10;

    printf("Before swapping:\n");
    printf("x = %d, y = %d\n", x, y);
```

```
// Call the swap function, passing addresses of x and y
swap(&x, &y);

printf("After swapping:\n");
printf("x = %d, y = %d\n", x, y);

return 0;}
```

Explanation:

- ***swap function:***

- It takes two integer pointers (int *a, int *b) as parameters. These pointers are used to directly modify the values of x and y in main.
- The *a and *b allow us to access the values stored at the addresses pointed to by a and b.
- The function swaps the values by using a temporary variable temp.

- ***In main:***

- We declare two variables x and y and initialize them with values 5 and 10 respectively.
- We then call the swap function, passing the addresses of x and y using the address-of operator &.

Output:

Before swapping:

x = 5, y = 10

After swapping:

x = 10, y = 5

Pointers and arrays:

Pointers and arrays in C are closely related concepts. In fact, in many cases, the name of an array can be treated as a pointer to its first element. Understanding how pointers and arrays interact is crucial for efficient memory management and accessing data structures in C.

1. Basic Concept / Array Name as a Pointer

In C, an array is a collection of elements of the same type stored in contiguous memory locations. While an array name (like arr) points to the first element of the array, it is not exactly the same as a pointer.

However, for many operations, arrays and pointers behave in similar ways. So, `arr` and `&arr[0]` essentially give the same address (the address of the first element of the array).

For example:

```
int arr[] = {10, 20, 30, 40};
```

```
int *ptr = arr; // This is equivalent to: int *ptr = &arr[0];
```

Here, `ptr` is a pointer that holds the address of the first element of the array `arr`.

2. Accessing Array Elements Using Pointers/ Pointer Arithmetic with Arrays

You can use pointers to access the elements of an array. The pointer can be incremented or decremented to move through the array.

You can use pointer arithmetic to navigate through an array. When you increment a pointer, it points to the next element of the array, considering the size of the data type the pointer points to.

Example of Pointer Arithmetic with Arrays:

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[] = {10, 20, 30, 40};
```

```
    int *ptr = arr; // Pointer to the first element of the array
```

```
    // Access elements using pointer arithmetic
```

```
    printf("%d\n", *(ptr + 0)); // Outputs 10 (arr[0])
```

```
    printf("%d\n", *(ptr + 1)); // Outputs 20 (arr[1])
```

```
    printf("%d\n", *(ptr + 2)); // Outputs 30 (arr[2])
```

```
    printf("%d\n", *(ptr + 3)); // Outputs 40 (arr[3])
```

```
    return 0;
```

```
}
```

Here, `*(ptr + i)` is equivalent to `arr[i]`, because `ptr + i` gives the address of `arr[i]`.

3. Arrays and Function Arguments

When you pass an array to a function, you're actually passing a pointer to the first element of the array. This means that if you modify the elements inside the function, it will affect the original array.

//Example programs to show passing array to function

```
#include <stdio.h>
```

```
// Function to modify array elements
void modifyArray(int *arr, int size) {
    for (int i = 0; i < size; i++) {
        arr[i] *= 2; // Double each element in the array
    }
}

int main() {
    int arr[] = {1, 2, 3, 4};
    int size = sizeof(arr) / sizeof(arr[0]);
    modifyArray(arr, size);

    // Print modified array
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]); }
    return 0;
}
```

In this example, the function modifyArray takes a pointer (int *arr) as its argument and modifies the original array by doubling each element.

Summary of Key Points

1. Array name as pointer: In many contexts, the name of an array is treated as a pointer to its first element.
2. Accessing array elements: You can use pointers to access array elements through dereferencing (*ptr) or pointer arithmetic (*(ptr + i)).
3. Function arguments: Arrays are passed to functions as pointers, allowing you to modify the original array directly.
4. Pointer arithmetic: You can perform arithmetic on pointers to traverse arrays, and C will automatically handle the size of the data type (e.g., ptr++ for an int * moves by sizeof(int)).

C – Pointer to Pointer (Double Pointer):

In C, double pointers are those pointers which stores the address of another pointer. The first pointer is used to store the address of the variable, and the second pointer is used to store the address of the first pointer. That is why they are also known as a pointer to pointer.

Declaration

A double pointer can be declared similar to a single pointer. The difference is we have to place an additional ‘*’ before the name of the pointer.

Syntax: `type **name;`

Above is the declaration of the double pointer with some name to the given type.

Initialization

The double pointer stores the address of another pointer to the same type.

Syntax: `name = &single_ptr;` *// initialization after declaration*

Syntax: `type **name = &single_ptr;` *// initialization with declaration*

Dereferencing

To access the value pointed by double pointer, we have to use the deference operator * two times.

`**name;` *// Gives you the value of the variable it points to*

Let's take a look at an example:

```
#include <stdio.h>
```

```
int main() {
```

```
    // Initialize a integer variable named var with value 10
```

```
    int var = 10;
```

```
    // declare and initialize a pointer ptr1 with address of var variable (&var)
```

```
    int *ptr1 = &var;
```

```
    // declare and initialize a double-pointer (ptr2) with address of pointer (ptr1)
```

```
    int **ptr2 = &ptr1;
```

```
    printf("var: %d\n", var); //var:10
```

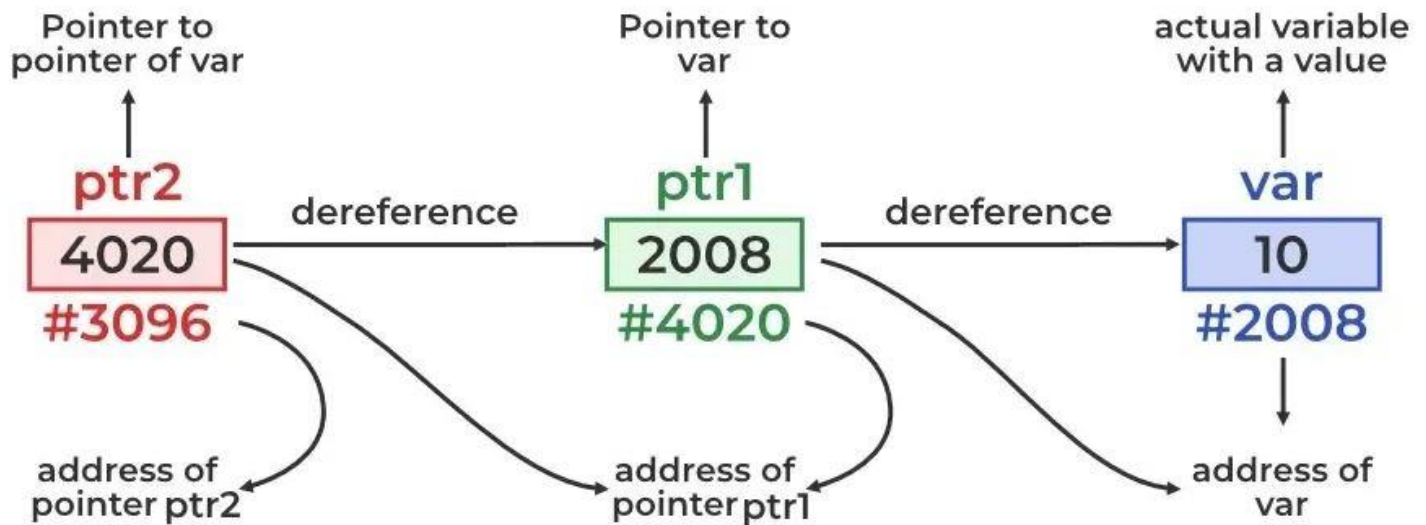
```
    printf("*ptr1: %d\n", *ptr1); //*ptr:10
```

```
    printf("**ptr2: %d", **ptr2); //**ptr2:10
```

```
    return 0;
```

```
}
```

Double Pointer



Multilevel Pointers in C

Double Pointers are not the only multilevel pointers supported by the C language. What if we want to change the value of a double pointer? In this case, we can use a triple pointer, which will be a pointer to a pointer to a pointer i.e, `int ***t_ptr`.

Examples:

```
int*** ptr3; // Triple pointer
```

```
int**** ptr4; // Quadruple Pointer
```

Similarly, to change the value of a triple pointer we can use a pointer to a pointer to a pointer to a pointer (Four level Pointer). In other words, we can say that to change the value of a "level - x" variable we can use a "level - x+1" pointer. And this concept can be extended further.

Note: We can use any level pointer in C. There is no restriction about it but it makes the program very complex and vulnerable to errors.

Dynamic Memory Allocation:

Dynamic Memory Allocation in C provides several functions to allocate, reallocate, and free memory at runtime, enabling flexible management of memory according to a program's needs. These methods are essential for creating data structures that can grow or shrink dynamically, such as linked lists or dynamic arrays.

There are four methods of Dynamic Memory Allocation in C.

1. Malloc:

The malloc() function in C, crucial for dynamic memory allocation, allocates a specified byte size on the heap and returns a void pointer to this space. Found in <stdlib.h>, it does not initialize the allocated memory, leaving it with garbage values. Cast the void pointer to the desired data type to use the allocated memory.

Syntax: The general syntax for malloc() in C involves casting the void pointer to the desired data type:

```
(cast-data-type *)malloc(size-in-bytes);
```

For instance, to allocate memory for an integer:

```
int *ptr = (int *)malloc(sizeof(int));
```

// C Program to dynamically allocate an array using malloc()

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {  
    char *ptr = (char *)malloc(sizeof(char)); // Allocates 1 byte of memory  
  
    if (ptr == NULL) {  
        printf("Memory Error!\n");  
    }  
    else {  
        *ptr = 'S'; // Assigns 'S' to the allocated memory  
        printf("%c", *ptr); // Prints 'S'  
    }  
    return 0;}
```

Output:

S

Explanation:

1. The program dynamically allocates 1 byte of memory sufficient for a char type and assigns it to ptr.
2. It checks if the memory allocation was successful (i.e., ptr is not NULL). If not, it prints an error message.
3. If successful, the program assigns the character 'S' to the allocated space and prints it.

2. Calloc:

calloc() in C is also a method that is used to allocate memory blocks in the heap section. Still, it is generally used to allocate a sequence of memory blocks (contiguous memory) like an array of elements. It is also present in <stdlib.h> header file.

General Syntax:

(cast-data-type *)calloc(num, size-in-bytes);

calloc() in C is a function that takes two arguments, first is the size of the array (number of elements) and second is the sizeof() data type (in bytes) of which we have to make an array.

Example:

```
int *arr = (int *)calloc(5, sizeof(int));
```

In the above statement, we are allocating an integer array of size 5 at runtime and assigning it to an integer pointer arr, arr will store the address of the very first element of the allocated array.

// C Program to dynamically allocate an array using calloc()

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {  
    int n=10;
```

```
// Dynamically allocated array of size 10 using calloc() array
```

```
//elements are initialized with 0 arr stores the base address of the memory block
```

```
    char *str = (char *)calloc(n, sizeof(char));
```

```
    if (str == NULL) {  
        printf("Memory Error!\n");}
```

```
else {  
    // initializing array with char variables  
    for (int i = 0; i < n; i++){  
        char ch;  
        scanf("%c", &ch);  
        *(str + i) = ch;}
```

```

        // printing array using pointer
        for (int i = 0; i < n; i++) {
            printf("%c", *(str + i));}
    return 0;}

```

3. free():

The free() function in C plays a critical role in memory management by deallocating memory blocks previously allocated with malloc() or calloc(). This helps prevent memory leaks and ensures efficient use of memory.

Defined in <stdlib.h>.

syntax for free():

```
void free(void* ptr);
```

where ptr is a pointer to the memory block to be freed. Example usage is as simple as free(ptr);.

// C Program to dynamically deallocate an array using free()

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

int main() {
    // Using malloc() for a single character
    char *ptr = (char *)malloc(sizeof(char));
    if (ptr == NULL) {
        printf("Memory Error!\n");
    } else {
        *ptr = 'S';
        printf("%c\n", *ptr); // Print the character
        free(ptr); // Deallocate memory
        ptr = NULL; // Avoid dangling pointer
    }
}

```

```
// Using calloc() for an array of characters
```

```
int n = 5; // Example array size
```

```
char *str = (char *)calloc(n, sizeof(char));
```

```

if (str == NULL) {
    printf("Memory Error!\n");
} else {

```

```
    // Example string "Hello"
```

```

    str[0] = 'H';
    str[1] = 'e';
    str[2] = 'l';
    str[3] = 'l';
    str[4] = 'o';
    for (int i = 0; i < n; i++) {
        printf("%c", str[i]); // Print the array
    }
    free(str); // Deallocate the array
    str = NULL; // Avoid dangling pointer
}
return 0;}

```

Output:

S
Hello

Explanation:

1. Initially, memory for a single character is allocated using malloc(), used to store the character 'S', and then deallocated with free().
2. The pointer is set to NULL afterwards to prevent it from becoming a dangling pointer.
3. Next, memory for an array of characters is allocated using calloc(), which initializes the allocated memory to zero.
4. The array is used to store and print the string "Hello". After use, the memory is deallocated using free(), and the pointer is set to NULL to avoid dangling pointers.

4. realloc():

The realloc() function in C is utilized for resizing a memory block previously allocated with malloc() or calloc(). This versatile function can either increase or decrease the allocated memory size and even allocate or deallocate memory.

Syntax:

```
void *realloc(void *ptr, size_t new_size);
```

- ptr is a pointer to the memory block to resize.
- new_size is the new memory size in bytes.

Example:

```
int *arr = (int *)realloc(arr, new_size * sizeof(int));
```

// C Program to dynamically allocate an array using realloc()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5; // Initial array size
    // Allocate array using malloc()
    int *arr = (int *)malloc(n * sizeof(int));

    // Fill array with values
    for (int i = 0; i < n; i++) {
        arr[i] = i + 1; }

    // Resize array to double its size
    arr = (int *)realloc(arr, 2 * n * sizeof(int));

    // Fill new part of array
    for (int i = n; i < 2 * n; i++) {
        arr[i] = i + 1;}

    // Print resized array
    for (int i = 0; i < 2 * n; i++) {
        printf("%d ", arr[i]);}

    // Free allocated memory
    free(arr);

    return 0;}
```

Output:

1 2 3 4 5 6 7 8 9 10

Explanation:

1. An array of integers is initially allocated with malloc() for 5 elements.
2. The realloc() function is then used to double the array's size.
3. The newly allocated half of the array is filled with consecutive integers.
4. The entire resized array is printed, showcasing the successful resize.
5. Finally, the dynamically allocated memory is freed using free().